hashlock.

2024

Security Audit

XBO (Token)

December 2024 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

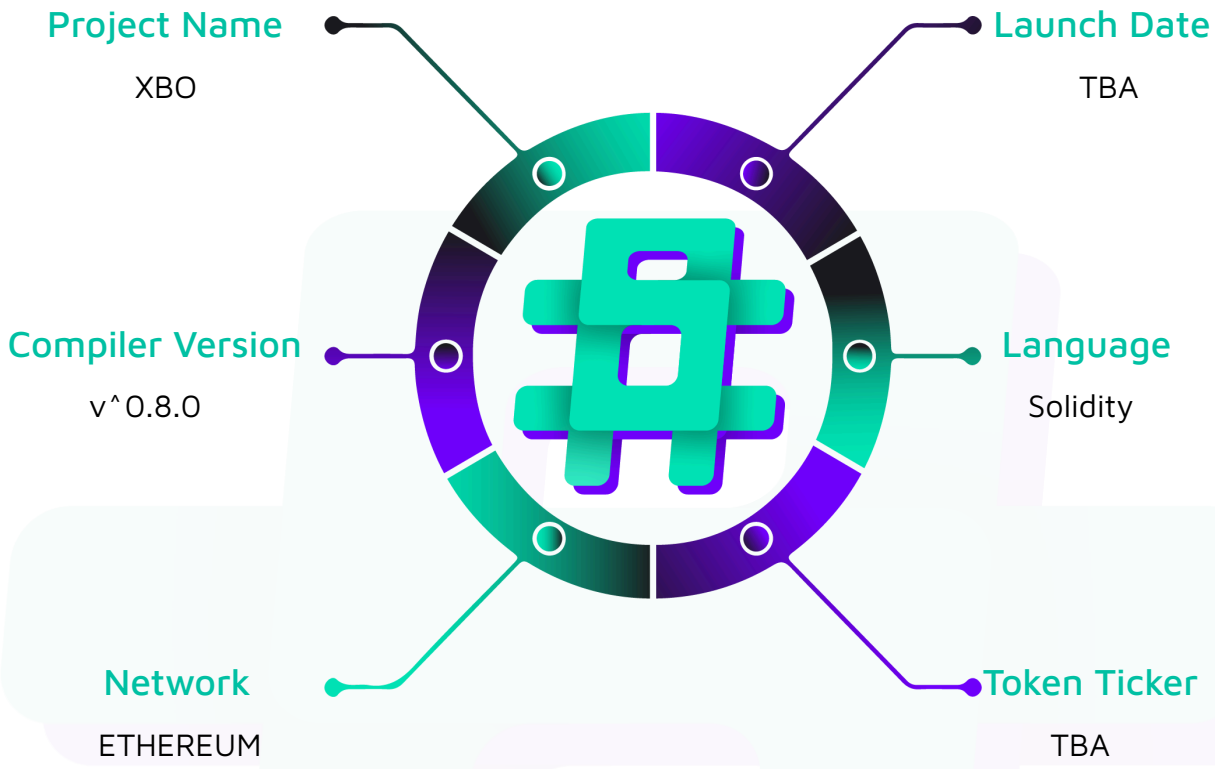
The XBO team partnered with Hashlock to conduct a security audit of their StandardToken.sol smart contract. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

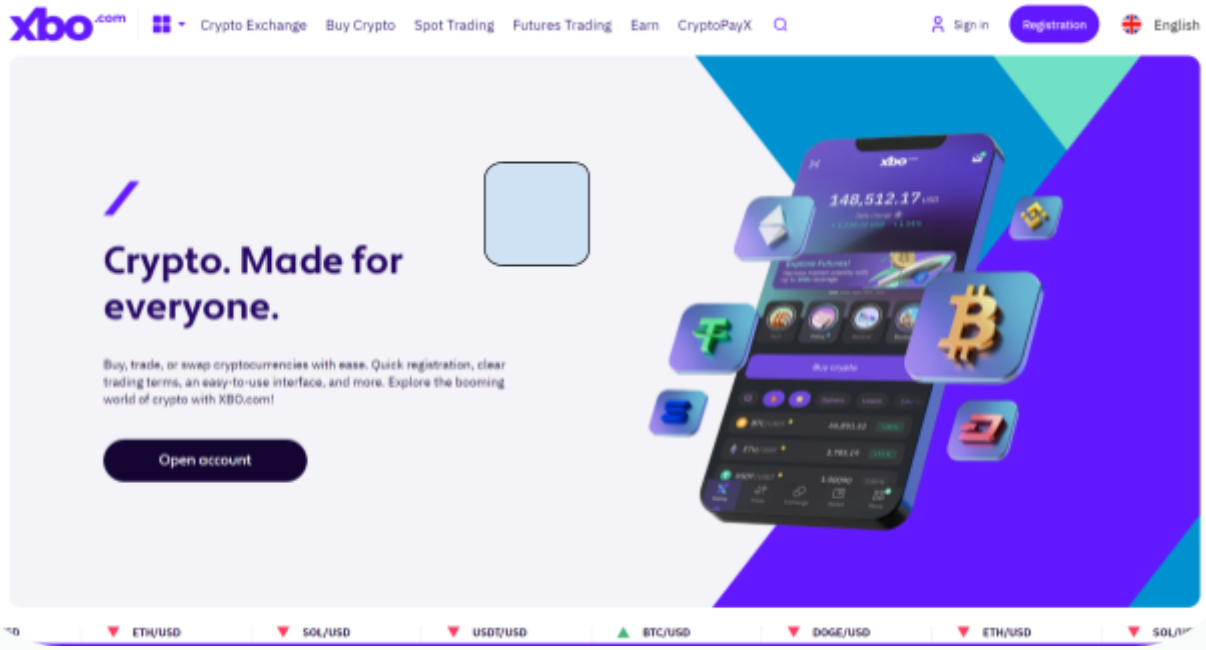
XBO.com is a user-centric cryptocurrency exchange platform that simplifies digital asset trading for both newcomers and seasoned investors. With a diverse selection of top-tier cryptocurrencies, including Bitcoin (BTC), Ethereum (ETH), and Solana (SOL), users can seamlessly buy, sell, and swap assets through an intuitive interface. The platform offers multiple payment methods, transparent trading conditions, and a user-friendly verification process, ensuring a smooth and secure experience. XBO.com prioritizes security by employing advanced technologies from AWS, Cloudflare, and Fireblocks, and complies with stringent regulations, including Anti-Money Laundering (AML) and General Data Protection Regulation (GDPR) standards. Recognized for its excellence, XBO.com was awarded "Best Digital Assets Exchange - Retail" at the 2024 Global Digital Assets Awards. Additionally, the platform features a loyalty program that rewards active users with benefits such as cashback on exchanges and discounted trading fees, enhancing the overall trading experience.

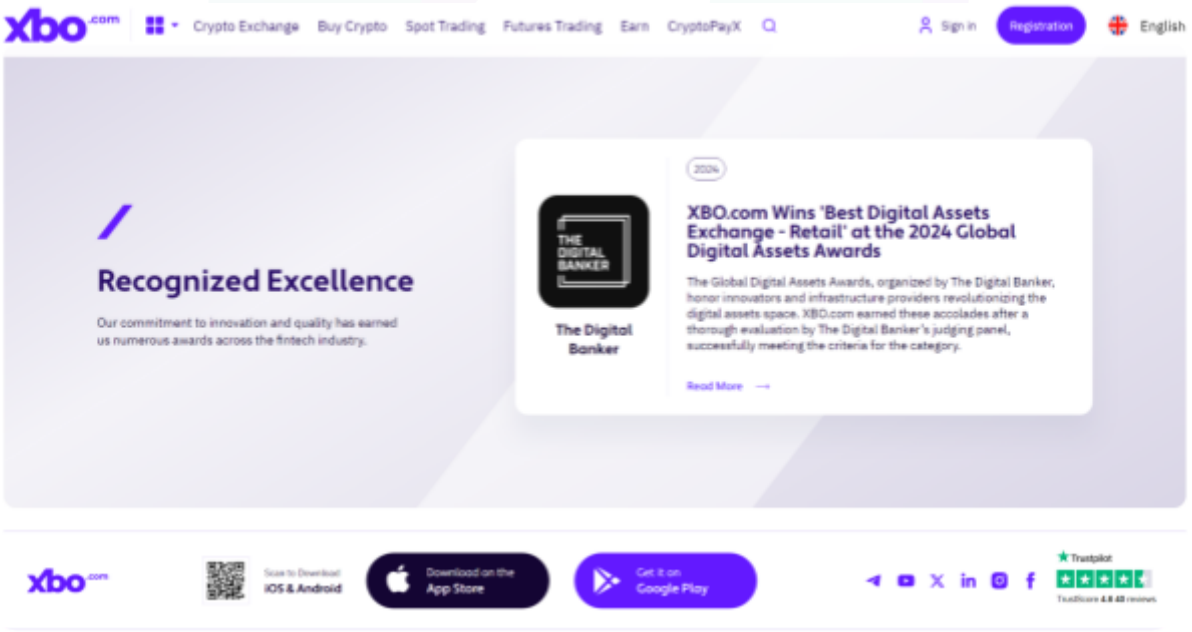
Project Name: XBO
Compiler Version: ^0.8.0
Website: https://www.xbo.com/
Logo:

Visualised Context:



Project Visuals:





Audit scope

We at Hashlock audited the solidity code within the XBO project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**XBO Protocol Smart Contracts**
Platform	**Ethereum / Solidity**
Audit Date	**December, 2024**
Contract 1	StandardToken.sol
Contract 1 Address	0x167e6fcd728cdd00d012d8862934c2c333cb0839

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that have since been acknowledged.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The general security overview is presented in the Standardised Checks section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been acknowledged.

Hashlock found:

3 Medium severity vulnerabilities

2 Low severity vulnerabilities

2 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour

StandardToken.sol
- Allows users to:
  - Transfer tokens to other addresses
  - Approve other addresses to spend tokens on their behalf
  - Transfer tokens from one address to another (if approved)
  - Check token balances and allowances
  - Increase or decrease token allowances
- Allows contract owner to:
  - Initialize the token with name, symbol, decimals, and total supply
  - Mint new tokens (internal function)
  - Burn tokens (internal function)
- Additional features:
  - Implements ERC20 standard interface
  - Upgradeable contract structure
  - Customizable number of decimals
  - Hook for custom logic before token transfers
	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the XBO project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the XBO project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] StandardToken#approve - Approval Race Condition

Description

1. The `approve` function in the contract allows a spender to use tokens on behalf of the owner. However, this implementation is vulnerable to a race condition known as the ERC20 approval race attack. The issue arises when allowances change:
2. Suppose an owner has already approved a certain number of tokens (`oldAllowance`) for a spender.
3. If the owner wishes to set a new allowance (e.g., reduce it), an attacker can front-run the change by using the transferFrom function to move the originally approved amount before the new transaction is mined.
4. This results in the spender being able to use allowances both before (`oldAllowance`) and after (`newAllowance`), causing unintended token transfers.

Impact

- Users who approve a new allowance for a spender can have their funds drained by an attacker because the attacker can exploit the time gap between the allowance change and the transaction confirmation.
- Loss of tokens for users attempting to adjust their allowance.

Recommendation

To mitigate this issue:

1. Replace approve with `increaseAllowance()` and `decreaseAllowance()`:
    a. These functions adjust the allowance incrementally instead of resetting it, reducing the risk of abuse during transaction racing.
    b. For instance:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    _approve(
        _msgSender(),
        spender,
        _allowances[_msgSender()][spender] + addedValue
    );
    return true;
}

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    uint256 currentAllowance = _allowances[_msgSender()][spender];
       require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    _approve(
        _msgSender(),
        spender,
        currentAllowance - subtractedValue
    );
    return true;
}
```

2. Alternative fix: Setting allowance to 0 before making adjustments:

   Tokens like USDT enforce this approach to prevent front-running attacks. Before updating an allowance to a new non-zero value, it must first be set to 0:

```
function approve(address spender, uint256 amount) public virtual override returns (bool) {
```

```
    _approve(_msgSender(), spender, 0);  // Reset allowance
    _approve(_msgSender(), spender, amount);  // Set new allowance
    return true;
}
```

3. Use OpenZeppelin's SafeERC20 library, which provides safe wrapper functions (`safeApprove`, `safeIncreaseAllowance`, etc.) to handle token approvals securely.

Note

The XBO team clarified that while their token may have certain vulnerabilities in very specific situations, these risks are minimal for their intended use. The token will primarily be used off-chain within their exchange or held in a Web3 wallet. They do not anticipate users connecting their wallets to decentralized finance (DeFi) platforms. However, in the event this does occur, they expect users to primarily interact with well-known and audited protocols, such as Uniswap, which are considered secure.

Status

Acknowledged

[M-02] StandardToken#_setupDecimals - Lack of Enforcement Modifier

Description

The `_setupDecimals` function is intended to set the token's decimal precision, typically during the contract's initialization. Without enforced restrictions, it can potentially be misused or invoked after the initialization phase, leading to unintended behavior such as:

- Alteration of Decimals: Changing the token's decimals after its deployment could disrupt integrations, user interfaces, and token-related calculations, leading to inconsistencies.
- Security Risk: Malicious actors or even legitimate maintainers could exploit this to alter token behavior unexpectedly.

```
function setupDecimals(uint8 decimals) internal virtual {
```

```
    decimals = decimals;

}
```

There is no modifier (such as initializer or `onlyOwner`) ensuring it cannot be called repeatedly or by unauthorized entities.

Recommendation

1. Restrict `_setupDecimals` Usage:

   Apply a modifier such as an initializer or require conditions to ensure the function is only invoked once during the initialization phase.

```
function _setupDecimals(uint8 decimals_) internal virtual {

    require(_decimals == 0, "Decimals already set");

    _decimals = decimals_;

}
```

2. Document Function Accessibility:

   Clearly document in the function definition or via contract comments when and how `_setupDecimals` may be used.

3. Consider Removal:

   If `_setupDecimals` is not required repeatedly, integrate its functionality directly within the initialize function instead of leaving it as a standalone method.

Status

Acknowledged

[M-03] StandardToken#transferFrom - Lack of Explicit Allowance Check

Description

In the provided ERC-20 implementation, the `transferFrom` function does not explicitly check whether the caller (`_msgSender()`) has sufficient allowance (`_allowances[sender][_msgSender()]`) before performing a transfer.

While Solidity 0.8+ protects against overflow and underflow by default, the function `implicitly` relies on subtraction-based reverts when an allowance is insufficient. However, such behavior lacks clarity for users and fails to return a clear error message, resulting in potential confusion and poor user experience when transactions fail.

Issues

1. Missing Explicit Allowance Check:
    a. There is no explicit required statement to ensure the allowance is sufficient.
    b. This results in unexpected reverts with generic error messages due to automatic arithmetic safeguards in Solidity.
2. Poor User Experience:
    a. Users receive generic "revert" transaction failure messages instead of a detailed error (e.g., "ERC20: transfer amount exceeds allowance"), which complicates debugging and integration with external systems.
3. Non-compliance with ERC-20 Best Practices:
    a. Standard implementations, such as OpenZeppelin's ERC-20, include clear checks for insufficient allowance before token transfers

Recommendation

Add the following explicit allowance check at the beginning of `transferFrom`:

```
require(
    _allowances[sender][_msgSender()] >= amount,
    "ERC20: transfer amount exceeds allowance"
);
```

Revised Implementation:

```
function transferFrom(
    address sender,
    address recipient,
    uint256 amount
```

```
) public virtual override returns (bool) {

    require(

        _allowances[sender][_msgSender()] >= amount,

        "ERC20: transfer amount exceeds allowance"

    );

    _transfer(sender, recipient, amount);

    _approve(

        sender,

        _msgSender(),

        _allowances[sender][_msgSender()] - amount

    );

    return true;

}
```

Status

Acknowledged

Low

[L-01] StandardToken#decreaseAllowance - No Checks for Allowance Decrease Below Zero

Description

The `decreaseAllowance` function does not explicitly check whether the `subtractedValue` exceeds the existing allowance before performing the subtraction, as shown in this line:

```
_allowances[_msgSender()][spender] - subtractedValue;
```

- This might cause an unexpected revert if the `subtractedValue` exceeds the current allowance.
- Due to Solidity 0.8's in-built overflow/underflow checks, the function will revert, but this might confuse users or external interfaces interacting with the contract.

Impact

- Allows unexpected reverts during allowance management which could disrupt user workflows (e.g., unintentionally blocking token spending or causing transaction failures).
- Does not explicitly safeguard intended behavior for interactions involving the allowance boundary.

Recommendation

Add an Explicit Bounds Check

Before adjusting the allowance, verify that the `subtractedValue` does not exceed the current allowance:

```
require(
    _allowances[_msgSender()][spender] >= subtractedValue,
    "ERC20: decreased allowance below zero"
);
```

Status

Acknowledged

[L-02] StandardToken#_mint&_burn - Missing Event Emissions for Updates

Description

The contract properly emits the Transfer event for balance-related changes (e.g., during `_mint` or `_burn`). However, it does not emit Approval events when the `_mint` or `_burn` functions update account balances.

Per the ERC20 standard's specification:

The Approval event is required for allowances (`approve`, `transferFrom`), but not for balance updates directly.

- Including Approval events with balance changes aids integration with certain applications or off-chain tools that reconstruct state based on events.

In the current contract:

- `_mint` increases `totalSupply` and the `balance` of the target account without emitting an Approval event.
- `_burn` decreases `totalSupply` and the `balance` of the target account without emitting an Approval event.

This omission might affect off-chain tools or services that track allowances via emitted events.

Recommendation

Emit Approval events in `_mint` and `_burn` to enhance compatibility and state tracking.

Proposed Code Changes:

For `_mint`:

```
function _mint(address account, uint256 amount) internal virtual {
    require(account != address(0), "ERC20: mint to the zero address");

    _beforeTokenTransfer(address(0), account, amount);

    _totalSupply += amount;
    _balances[account] += amount;
    emit Transfer(address(0), account, amount);

    // Emit Approval event for consistency
    emit Approval(account, address(0), _balances[account]);
```

```
}
```

For `_burn`:

```
function _burn(address account, uint256 amount) internal virtual {

    require(account != address(0), "ERC20: burn from the zero address");



    _beforeTokenTransfer(account, address(0), amount);



    _balances[account] -= amount;

    _totalSupply -= amount;

    emit Transfer(account, address(0), amount);



    // Emit Approval event for consistency

    emit Approval(account, address(0), _balances[account]);

}
```

Status

Acknowledged

QA

[Q-01] StandardToken- Lack of Zero Amount Checks in `transfer`, `transferFrom`, and `approve` Functions

Description

The provided implementation of the `transfer`, `transferFrom`, and `approve` functions lacks explicit checks for zero-value transactions or approvals. While this behavior is compliant with the ERC20 standard (which allows zero-value operations), including

checks for zero values can enhance gas efficiency and prevent unnecessary state changes. For example

- Zero-value approvals reassign the same value, resulting in unnecessary writes to storage and wasted gas.
- Zero-value transfers accomplish nothing but still consume gas for emitting events and modifying storage.

This issue, while not critical, can lead to inefficiencies if the functions are repeatedly called with zero values

Recommendation

To mitigate this inefficiency, add explicit zero-value checks in affected functions. Below are the recommended updates:

Updated `transfer` Function:

```
function transfer(address recipient, uint256 amount) public virtual override returns (bool) {

    require(amount > 0, "ERC20: transfer amount cannot be zero");

    _transfer(_msgSender(), recipient, amount);

    return true;

}
```

Updated `approve` Function:

```
function transfer(address recipient, uint256 amount) public virtual override returns (bool) {

    require(amount > 0, "ERC20: transfer amount cannot be zero");

    _transfer(_msgSender(), recipient, amount);

    return true;

}
```

Updated `transferFrom` Function:

```
function transferFrom(
    address sender,
    address recipient,
    uint256 amount
) public virtual override returns (bool) {
    require(amount > 0, "ERC20: transferFrom amount cannot be zero");
    _transfer(sender, recipient, amount);
    _approve(
        sender,
        _msgSender(),
        _allowances[sender][_msgSender()] - amount
    );
    return true;
}
```

Status

Acknowledged

[Q-02] StandardToken- Unused TokenType Enum

Description

The `TokenType` enum is defined in the contract but is completely unused throughout the implementation. While this issue does not impact the functionality or security of the contract, it adds unnecessary complexity and increases the compiled contract size. Unused code can lead to confusion for developers and auditors, making the contract harder to maintain and understand.

Recommendation

Remove the `TokenType` Enum

If there are no future plans to utilize the `TokenType` enum, removing it entirely would simplify the contract, reduce gas costs for deployment, and make the codebase cleaner.

```
enum TokenType {
    Standard,
    Liquidity,
    LiquidityFee,
    LiquidityBuySellFee,
    Burn,
    Baby,
    StandardAntiBot,
    LiquidityAntiBot,
    LiquidityFeeAntiBot,
    LiquidityBuySellFeeAntiBot,
    BurnAntiBot,
    BabyAntiBot
}
```

Removing this would both help decrease size and improve readability.

Status

Acknowledged

Centralisation

The XBO project values community-centered and transparency over centralisation.

XBO emphasizes a community-centric approach while maintaining transparency. Although some critical functions remain centralized for enhanced security and functionality, the project strives to balance decentralization with internal team accountability. This blend ensures operational security while keeping the protocol aligned with decentralized values.

Conclusion

After Hashlock's analysis, the XBO project seems to have a sound and well-tested code base, now that our vulnerability findings have been acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au